

CreditSense: Loan Risk Assessment Challenge

Written Report

Team name	Johnny-Izhak-Kyriakos
Student names	Johnny, Izhak, Kyriakos
Kaggle username(s)	JohnnyC0rp
Public-best pipeline	meta_overnight
Public-best local combined score	0.8495
Strongest local validation score	0.8576

1 Data Exploration and Preprocessing

The assignment asks for two outputs per applicant: a five-class `RiskTier` prediction and a continuous `InterestRate` estimate, scored by the equally weighted average of accuracy and R^2 [3]. The working dataset contains 35,000 labeled applications, 15,000 test rows, and 55 raw features that mix demographics, income, liabilities, credit history, and loan request context.

The first lesson from exploration was that missing values were informative instead of random noise. Property-related fields were often absent for renters, while some debt fields vanished because the customer simply did not hold that liability. Treating all missingness as the same would hide that semantic structure, so we explicitly encoded missing-value flags before any modeling. Class balance was reasonably healthy, which made validation accuracy stable enough to compare model families without heroic reweighting.



Figure 1: Initial exploration highlighted both class balance and structured missingness.

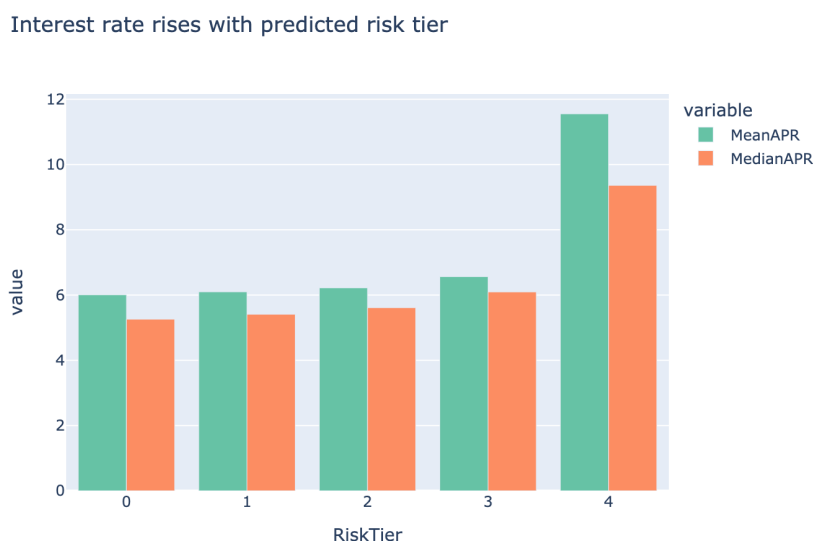


Figure 2: Interest rates rise monotonically with risk tier, confirming that the two targets are related but not identical.

Our preprocessing pipeline stayed intentionally conservative: median filling for numeric columns, ordinal encoding for categoricals, and frequency features for each categorical value. On top of that base, we added finance-specific ratios such as loan-share-of-income, debt pressure, delinquency severity, and derogatory burden. We also added a small controlled set of polynomial interactions so the tree models could capture sharp affordability cliffs without blowing the feature space into nonsense.

2 Feature Engineering

The best features were not generic transformations; they were domain features that translated banking intuition into measurable risk. We grouped the final engineered features into four families:

- **Affordability ratios:** loan share of income, monthly payment share, debt pressure, and asset coverage.
- **Delinquency severity:** weighted 30/60/90-day late-payment counts and their interactions with utilization and debt ratios.
- **Derogatory burden:** count and severity aggregates for charge-offs, collections, bankruptcies, and public records.
- **Semantic missingness:** borrower-context flags such as renter-without-property-value or owner-without-mortgage-balance.

Table 1 shows how the project score moved as we layered these ideas into the pipeline. The biggest jump came from moving beyond basic preprocessing into finance-aware ratios and controlled interaction terms. Later gains were smaller, but still meaningful enough to matter on the leaderboard.

Addition	Combined score	Why it helped
Domain ratios + semantic missingness	0.8225	Added affordability, utilization, and context-aware missingness signals.
Controlled polynomial interactions	0.8251	Captured non-linear stress interactions without overgrowing the feature space.
Tree ensemble diversification	0.8495	Different tree families corrected different blind spots.
Target statistics + calibrated bagging	0.8557	Injected supervised structure into sparse categorical groups.
Neural branch + weighted fusion	0.8576	Added smooth non-linear structure on top of the tree family.

Table 1: Major feature-and-model milestones during the project.

The classification and regression tasks shared several strong signals, but the ranking was not identical. Figure 3 shows that delinquency severity and utilization interactions dominate the class prediction task, while affordability and balance-sheet coverage become relatively more important for interest-rate prediction. That difference is exactly what we would expect in lending: default risk and pricing are related, but pricing also reacts to continuous affordability details.

Top engineered features for classification vs regression

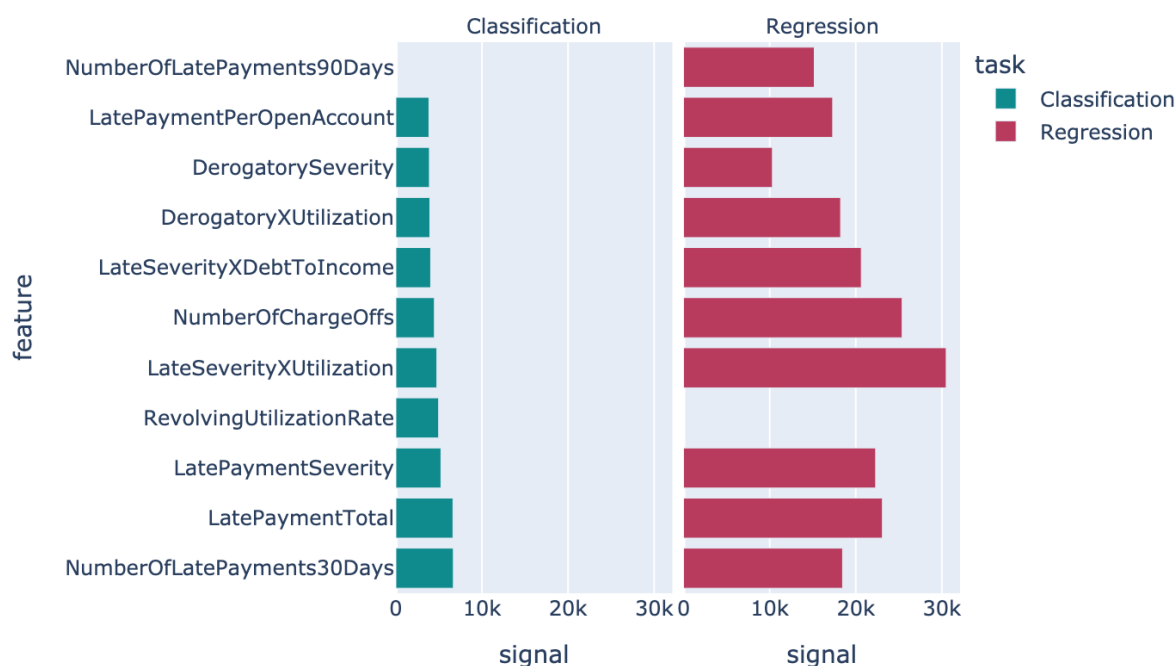


Figure 3: Top engineered features differ slightly between the classification and regression tasks.

3 Model Selection and Tuning

We evaluated both simple and complex model families. Linear baselines were useful for orientation but quickly saturated. ExtraTrees gave a strong non-boosting reference, then XGBoost, LightGBM, and CatBoost supplied the main performance gains through stronger non-linear partitions and better handling of mixed-scale tabular signals [1, 2, 4]. The final public-best Kaggle pipeline, `meta_overnight`, combined five branches:

1. XGBoost on the engineered numeric feature matrix.
2. LightGBM on the same preprocessed matrix.
3. CatBoost on the engineered frame with categorical columns preserved.
4. ExtraTrees as a low-bias diversity branch.
5. A compact MPS neural branch that contributed a different error profile.

Instead of retuning everything inside the public notebook, we reused the historically tuned branch settings and the previously learned blend/meta hyperparameters. This makes the notebook readable and reproducible while preserving the public-best model family that actually mattered on Kaggle.

Model	Accuracy	R^2	Combined
meta_overnight (public best)	0.8477	0.8513	0.8495
blend_overnight	0.8424	0.8485	0.8455
target_stats_blend	0.8563	0.8552	0.8557
mlp_meta_stack_v1	0.8587	0.8556	0.8572
mlp_weighted_blend_v1 (local best)	0.8600	0.8553	0.8576

Table 2: Representative validation leaderboard from the project history.

To reduce overfitting, we used a fixed stratified holdout split, explicit seeds, early stopping in the neural branch, and conservative tree regularization. We also resisted the temptation to submit every local improvement: the strongest local blend (`mlp_weighted_blend_v1`) was excellent for validation analysis, but it relied on a deeper stack of nested branches and tiny residual weights. For the public repo and the final report, `meta_overnight` is the right pipeline to present because it is the cleanest strong submission path and the best match to the Kaggle story.

Takeaway. The project ended with two truths that can coexist peacefully: the `meta_overnight` ensemble is the best public, report-ready Kaggle pipeline, while later local experimentation pushed validation a little higher through more intricate stacking. The public repo therefore emphasizes clarity and reproducibility, and the report keeps the later local gains as evidence of what we learned rather than what we chose to publish.

References

- [1] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [2] Guolin Ke et al. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 2017.
- [3] MBZUAI. Ai1215 — creditsense: Loan risk assessment challenge, 2026. Course project assignment sheet.
- [4] Liudmila Prokhorenkova et al. Catboost: Unbiased boosting with categorical features. *Advances in Neural Information Processing Systems*, 2018.